Serialization and Deserialization for Backend Engineers

1. Executive Summary

The core thesis of this video explores how isolated, structurally different computing environments (e.g., a dynamic JavaScript client and a strongly-typed Rust server) successfully exchange and interpret data across a network. **Serialization and deserialization** serve as the standardized bridge, converting in-memory data structures into a universally readable format for transmission, and reconstructing them upon receipt. By abstracting away the low-level complexities of the OSI model's physical and transport layers, backend engineers can rely on common serialization formats like JSON to ensure seamless, language-agnostic communication in modern HTTP/REST architectures.

---

2. Deep-Dive Technical Content

The Cross-Language Communication Problem

In a typical client-server architecture, frontends and backends are often built using entirely different technology stacks.

- **The Client**: Usually a browser running a dynamic JavaScript framework (React, Angular, Vue).
- **The Server**: Running locally or remotely (AWS, GCP, Azure), often written in a strict, compiled language like Rust or Java.

Because these languages handle data types and memory differently, direct transmission of native objects is impossible. To solve this, a common standard format is agreed upon. Data is converted into this standard format before transmission and parsed back into native data types (e.g., a Rust `struct` or a Java `Class`) upon arrival.

The OSI Model Context (A Backend Engineer's Mental Model)

While data technically travels down the **OSI Model**—originating at the Application layer, transforming into data frames, IP packets, and eventually physical bits (0s and 1s transmitted via voltage signals over optical fiber)—backend engineers do not need to manage these intermediary steps.

- **The Abstraction**: Your responsibility lies purely at the **Application Layer**. You serialize native objects into JSON on the sending side and deserialize JSON back into native objects on the receiving side. The network stack handles the rest.

Serialization Standards Landscape

Serialization standards fall into two primary categories:

1. Text-Based Serialization

Human-readable formats predominantly used for HTTP/REST communications.

- **JSON (JavaScript Object Notation)**: The industry standard (roughly 80% of HTTP communication). It is domain and language-agnostic despite its naming origins. Heavily used in REST APIs, configuration

files, and logging.
- **YAML**: Often used for configurations.
- **XML**: The legacy standard, heavily reliant on tags.

2. Binary Format Serialization

Machine-readable formats optimized for performance and lower bandwidth, commonly used in gRPC or internal microservice communications.

- **Protobuf (Protocol Buffers)**: A highly efficient binary format popularized by Google.

Deep-Dive into JSON

JSON enforces strict syntactic rules for data transmission:

- **Braces**: Objects must be enclosed in `{}`.
- **Keys**: Must be strictly wrapped in **double quotes** (`"key"`).
- **Values**: Can only be specific types: `String`, `Number`, `Boolean`, `Array`, or a `Nested Object`.

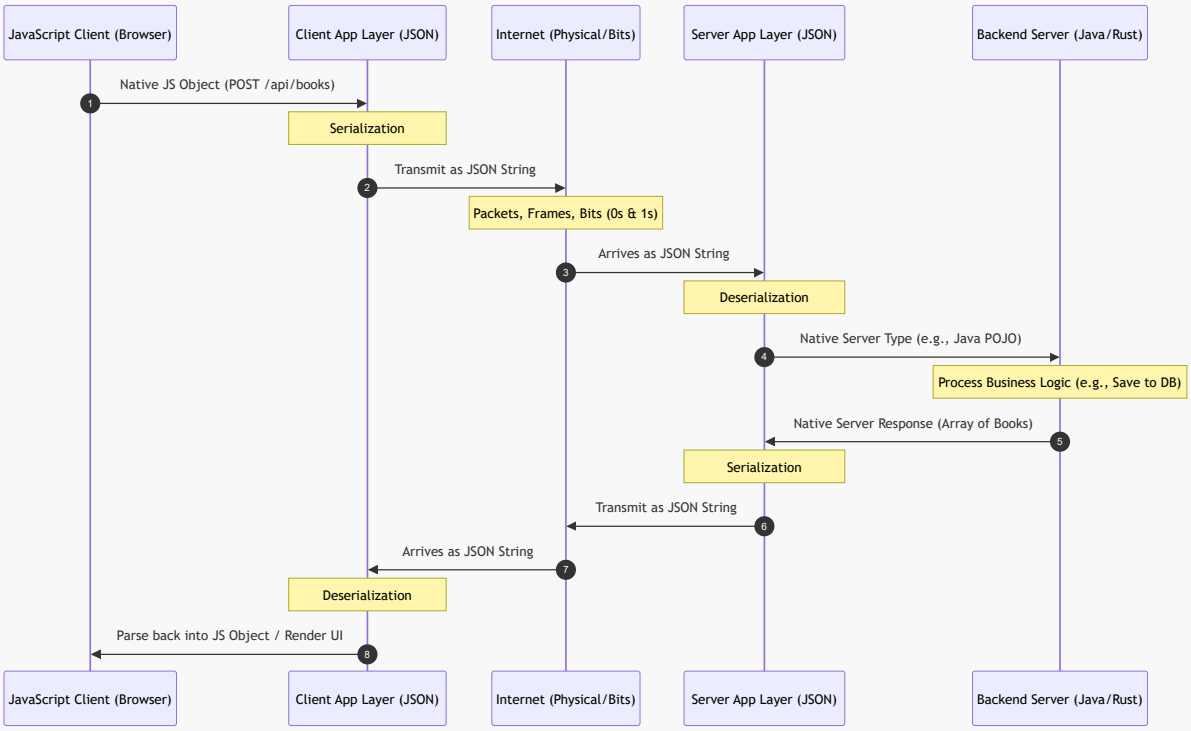
During the demo (using Burp Suite), a `POST` request was made to `/api/books` sending a JSON payload containing `id`, `title`, and `author`. The server digested this payload and responded with an array containing the newly updated list of book objects.

---

3. Code & Architecture Visualizations

Data Flow Architecture

The following Mermaid.js sequence diagram illustrates the lifecycle of serialized data traversing the OSI model conceptually, moving between a JavaScript Client and a Backend Server.



Java Implementation: Serialization & Deserialization

Though the video highlighted JavaScript and Rust, the underlying principles apply universally. Below is the exact implementation of the video's `/api/books` payload (including the nested object example mentioned) using Java and the popular `Jackson` library.

```
import com.fasterxml.jackson.databind.ObjectMapper;
import java.util.List;
import java.util.ArrayList;

// 1. Define the Native Data Structures (POJOs)
class Book {
    public int id;
    public String title;
    public String author;
    public Address publisherAddress; // Nested Object Example

    // Default constructor required for Jackson Deserialization
    public Book() {}

    public Book(int id, String title, String author, Address publisherAddress) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.publisherAddress = publisherAddress;
    }
}

class Address {
```

```
    public String country;
    public long phoneNumber;

    public Address() {}
    public Address(String country, long phoneNumber) {
        this.country = country;
        this.phoneNumber = phoneNumber;
    }
}

public class SerializationDemo {
    public static void main(String[] args) {
        try {
            ObjectMapper mapper = new ObjectMapper();

            // --- SERIALIZATION (Server to Client) ---
            Address address = new Address("India", 9876543210L);
            Book newBook = new Book(1, "Backend Engineering", "Sriniously",
address);

            // Convert Java Object into JSON String
            String jsonResponse = mapper.writeValueAsString(newBook);
            System.out.println("Serialized JSON to send over network:\n" +
jsonResponse);

            // --- DESERIALIZATION (Client to Server) ---
            // Simulating an incoming JSON string from an HTTP POST request body
            String incomingJson = "{\"id\":2,\"title\":\"System
Design\",\"author\":\"Alex\",\"publisherAddress\":
{\"country\":\"USA\",\"phoneNumber\":123456789}}";

            // Convert JSON String back into a Java Object
            Book receivedBook = mapper.readValue(incomingJson, Book.class);
            System.out.println("\nDeserialized Native Java Object:");
            System.out.println("Title: " + receivedBook.title + " | Country: " +
receivedBook.publisherAddress.country);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

---

4. Key Metrics & Comparisons

The video mentions various technologies and serialization formats. Here is how they benchmark against each other in backend architecture:

Feature / Metric	JSON	XML	YAML	Protobuf (Binary)
Format Type	Text-based	Text-based	Text-based	Binary
Human Readable?	Yes	Yes (but verbose)	Yes (highly readable)	No
Primary Use Case	REST APIs, Client-Server HTTP	Legacy Enterprise Systems, SOAP	Configuration Files (e.g., Docker, K8s)	Microservices, gRPC, High-performance apps
Data Types Supported	String, Num, Bool, Array, Object	Strings (requires custom parsing for others)	Complex hierarchical data	Strongly typed (requires schema definitions)
Parsing Speed	Fast	Slow	Moderate	**Extremely Fast**
Payload Size	Moderate	Large (heavy tags)	Moderate	**Minimal / Highly Compressed**

5. Actionable Takeaways

- **Abstract Network Layers out of your Code:** Treat the network as a black box. Focus strictly on proper application-layer mapping (JSON to Object, Object to JSON). Let the OS handle packets, frames, and voltages.
- **Enforce Strict Typing on the Backend:** Because clients (like JavaScript) are often dynamically typed, use strict data transfer objects (DTOs) in your compiled backend (Java/Rust) to validate incoming JSON strictly.
- **Standardize JSON Keys:** Always use double quotes `""` for keys. Attempting to pass single quotes or unquoted keys will cause serialization exceptions on strict backend parsers.
- **Match the Protocol to the Problem:** Default to HTTP/REST and JSON for general client-to-server communication due to widespread tooling and support. Keep gRPC/Protobuf in mind exclusively for internal server-to-server microservice communications where bandwidth/parsing bottlenecks exist.
- **Leverage Robust Libraries:** Never write custom JSON string-parsing regex. Utilize production-hardened libraries (e.g., `Jackson` or `Gson` in Java, `serde` in Rust) to handle edge cases, escaping, and secure deserialization.